UNIVERSITATEA [NUMELE UNIVERSITĂȚII]

Facultatea de [Facultate]

Departamentul de [Departament]

Programul de master: [Denumire program]

Sistem E-Medicine pentru Analizarea și Monitorizarea Datelor Epidemiologice bazat pe Modelul Matematic SIR

LUCRARE DE DISERTAȚIE

Coordonator științific:

Prof. Dr. [Nume Coordonator]

[Titlul / Funcția]

Absolvent:

[Prenume Nume]

Seria [an], Grupa [grupă]

[Orașul], 2026

Rezumat

Lucrarea de față prezintă proiectarea și realizarea unui sistem e-medicine dedicat analizării și monitorizării datelor epidemiologice, având ca fundament matematic modelul SIR (Susceptibili–Infectați–Recuperați). Modelul este implementat și testat comparativ în patru limbaje de programare cu utilizare intensă în calculul științific: R, Julia, Python și SAS.

Implementările sunt evaluate pe mai multe configurații hardware, urmărindu-se comportamentul algoritmilor din perspectiva performanței și a scalabilității în funcție de dimensiunea seturilor de date procesate. Datele epidemiologice utilizate sunt stocate într-o bază de date distribuită YugabyteDB, rulată într-un cluster Kubernetes, asigurând disponibilitate ridicată, toleranță la erori și distribuție eficientă a datelor.

Pentru centralizarea, compararea și vizualizarea rezultatelor este dezvoltată o aplicație web dedicată, compusă dintr-un API backend implementat în limbajul Go și o interfață frontend realizată în Vue.js. Aceasta oferă o perspectivă unificată asupra performanței și comportamentului modelelor epidemiologice analizate, permițând consultarea rezultatelor rulărilor, a seturilor de date utilizate și a graficelor asociate.

Cuvinte cheie: model SIR, epidemiologie computațională, e-medicine, calcul comparativ, YugabyteDB, Kubernetes, Go, Vue.js, benchmark de performanță.

Cuprins

Rezumat **i**

Listă de figuri **vii**

Listă de tabele **viii**

Lista algoritmilor **ix**

1 Introducere **1**

1.1 Context și motivație 1
1.2 Obiectivele lucrării 2
1.3 Contribuții originale 2
1.4 Structura lucrării 3

2 Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase **4**

2.1 Fundamente de epidemiologie 4
2.1.1 Definiții și concepte fundamentale 4
2.1.2 Parametrii fundamentali ai transmiterii 5
2.2 Modele compartimentale în epidemiologie 5
2.2.1 Cadrul general 5
2.2.2 Modelul SI 6
2.2.3 Modelul SIS 6
2.2.4 Modelul SEIR 7
2.2.5 Modelul SIRS 7
2.3 Modelul SIR — Analiză Matematică Detaliată 7
2.3.1 Ipoteze și derivarea ecuațiilor din principii prime 7
2.3.2 Proprietăți matematice fundamentale 8
2.3.3 Analiza echilibrelor și stabilitate 10
2.3.4 Numărul de reproducere de bază $\mathcal{R}_0$ 11
2.3.5 Pragul de imunizare în turmă (herd immunity) 12
2.3.6 Ecuația dimensiunii finale a epidemiei 13

2.3.7 Extensii ale modelului SIR 14
2.4 Metode Numerice pentru Ecuații Diferențiale Ordinare 15
2.4.1 Clasificarea metodelor numerice 15
2.4.2 Metoda Euler explicită 15
2.4.3 Metoda Runge-Kutta de ordinul 4 (RK4) 16
2.4.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince) . . . 17
2.4.5 Stabilitate numerică și probleme stiff 19
2.5 Date Epidemiologice — Surse și Preprocesare 19
2.5.1 Surse de date epidemiologice 19
2.5.2 Structura datelor și preprocesare 20
2.5.3 Estimarea parametrilor din date reale 20

3 Tehnologii și Instrumente Utilizate 21
3.1 Limbaje de programare pentru calcul științific 21
3.1.1 R 21
3.1.2 Julia 22
3.1.3 Python 23
3.1.4 SAS 25
3.1.5 Analiză comparativă a limbajelor 26
3.2 Baze de date distribuite — YugabyteDB 26
3.2.1 Concepte de baze de date distribuite 26
3.2.2 Arhitectura YugabyteDB 27
3.2.3 Sharding și replicare 27
3.3 Orchestrarea containerelor — Kubernetes 28
3.3.1 Containere și nevoia de orchestrare 28
3.3.2 Arhitectura Kubernetes 28
3.3.3 Resurse Kubernetes relevante 29
3.4 Backend API — Go 29
3.4.1 Caracteristici și filosofia limbajului Go 29
3.4.2 Ecosistem pentru API-uri REST 30
3.5 Frontend — Vue.js 30
3.5.1 Arhitectura Vue 3 și Composition API 30
3.5.2 Ecosistem și biblioteci utilizate 30

4 Arhitectura Sistemului E-Medicine 32
4.1 Vedere de ansamblu 32
4.2 Cerințe funcționale și non-funcționale 32
4.2.1 Cerințe funcționale 32
4.2.2 Cerințe non-funcționale 32
4.3 Arhitectura bazei de date distribuite 33

4.3.1 Schema relațională . . . 33
4.3.2 Modelul datelor epidemiologice . . . 33
4.3.3 Strategia de partitionare și replicare în YugabyteDB . . . 33
4.4 Arhitectura API-ului backend Go . . . 33
4.4.1 Structura modulară a proiectului . . . 33
4.4.2 Diagrama endpointurilor REST . . . 33
4.4.3 Integrarea cu runner-ele de algoritmi . . . 33
4.5 Arhitectura frontend Vue.js . . . 34
4.5.1 Structura componentelor . . . 34
4.5.2 Fluxul de date și state management . . . 34
4.6 Infrastructura Kubernetes . . . 34
4.6.1 Topologia cluster-ului . . . 34
4.6.2 Deployment YugabyteDB pe Kubernetes . . . 34
4.6.3 Deployment API Go și frontend Vue.js . . . 34

5 Implementarea Modelului SIR 35
5.1 Seturile de date utilizate . . . 35
5.2 Implementarea în R . . . 35
5.2.1 Structura proiectului și dependențe . . . 35
5.2.2 Rezolvarea numerică cu pachetul deSolve . . . 35
5.2.3 Estimarea parametrilor din date reale . . . 35
5.2.4 Vizualizare și export rezultate . . . 35
5.3 Implementarea în Julia . . . 36
5.3.1 Structura proiectului și pachete . . . 36
5.3.2 DifferentialEquations.jl — definiție și solver-e . . . 36
5.3.3 Optimizări specifice limbajului Julia . . . 36
5.4 Implementarea în Python . . . 36
5.4.1 Structura proiectului și dependențe . . . 36
5.4.2 scipy.integrate.solve_ivp . . . 36
5.4.3 Vectorizare și optimizare NumPy . . . 36
5.5 Implementarea în SAS . . . 37
5.5.1 PROC IML — implementare completă . . . 37
5.5.2 Particularități față de celelalte implementări . . . 37
5.6 Interfața comună de integrare cu API-ul Go . . . 37

6 Evaluarea Performanței și Scalabilității 38
6.1 Metodologia de benchmarking . . . 38
6.1.1 Configurații hardware utilizate . . . 38
6.1.2 Metrici urmărite . . . 38
6.1.3 Protocoale de testare și reproductibilitate . . . 38

6.2 Scalabilitate în funcție de dimensiunea setului de date 38
6.3 Comparație timp de execuție — R vs Julia vs Python vs SAS 39
6.4 Consum de memorie și resurse 39
6.5 Acuratețea numerică a rezultatelor 39
6.6 Performanța bazei de date distribuite 39
6.6.1 Latență la scriere și citire 39
6.6.2 Comportament sub load — teste de stress 39
6.7 Performanța API-ului Go 39
6.8 Analiză comparativă sintetică 40

7 Aplicația Web E-Medicine 41
7.1 Descrierea funcționalităților aplicației 41
7.2 Backend Go — Implementare detaliată 41
7.2.1 Structura proiectului Go 41
7.2.2 Implementarea endpointurilor REST 41
7.2.3 Gestionarea execuției algoritmilor 41
7.2.4 Conexiunea la YugabyteDB 41
7.2.5 Autentificare și securitate 42
7.3 Frontend Vue.js — Implementare detaliată 42
7.3.1 Structura componentelor și pagini 42
7.3.2 Dashboard epidemiologic 42
7.3.3 Vizualizarea graficelor SIR 42
7.3.4 Comparatorul de rezultate între limbaje 42
7.3.5 Interfața pentru seturile de date 42
7.4 Fluxuri de utilizare (use cases) 42
7.5 Testare și validare funcțională 43

8 Concluzii și Direcții de Dezvoltare Viitoare 44
8.1 Sinteza contribuțiilor 44
8.2 Concluzii privind implementările modelului SIR 44
8.3 Concluzii privind infrastructura distribuită 44
8.4 Limitări ale lucrării 44
8.5 Direcții de dezvoltare viitoare 45

Bibliografie 46

A Cod Sursă — Implementarea R 48

B Cod Sursă — Implementarea Julia 49

C Cod Sursă — Implementarea Python 50

D Cod Sursă — Implementarea SAS **51**

E Schema Bazei de Date **52**

F Configurații Kubernetes **53**

G Documentația API REST **54**

Listă de figuri

2.1 Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare. . . . 8

Listă de tabele

3.1 Comparație a limbajelor de programare utilizate 26

Lista algoritmilor

2.1 Metoda Euler pentru sistemul SIR 16
2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR 17

Capitolul 1

Introducere

1.1 Context și motivație

Bolile infecțioase reprezintă una dintre cele mai vechi și persistente provocări ale umanității. De la pandemia de gripă din 1918 la epidemia de SARS din 2003 și, mai recent, pandemia de COVID-19 declanșată în 2019, evenimentele epidemice de amploare au demonstrat în mod repetat capacitatea lor de a perturba profund structurile sociale, economice și sanitare ale societăților moderne [1, 2].

Înțelegerea și controlul dinamicii bolilor infecțioase nu constituie doar o problemă medicală, ci reprezintă o provocare interdisciplinară care îmbină epidemiologia, matematica aplicată, informatica și știința datelor. La baza acestei înțelegeri stau *modelele matematice compartimentale*, instrumente formale care permit cuantificarea proceselor de transmitere, predicția evoluției epidemiilor și evaluarea intervențiilor de sănătate publică.

Modelul **SIR** (Susceptibili-Infectați-Recuperați), introdus de Kermack și McKendrick în lucrarea fondatoare din 1927 [3], rămâne și astăzi fundamentul teoretic al epidemiologiei matematice. Simplitatea sa elegantă — trei ecuații diferențiale ordinare cuplate — ascunde o bogăție matematică remarcabilă și o putere predictivă validată de zeci de ani de aplicații empirice.

Odată cu creșterea exponențială a volumului de date epidemiologice disponibile și cu diversificarea instrumentelor computaționale moderne, apare necesitatea unor *sisteme informatice integrate* capabile să gestioneze atât stocarea distribuită a datelor, cât și rularea eficientă a modelelor matematice și vizualizarea rezultatelor. Alegerea limbajului de programare și a arhitecturii software are un impact direct și semnificativ asupra performanței, scalabilității și mentenabilității unui astfel de sistem.

Lucrarea de față se înscrie în această direcție, propunând proiectarea și realizarea unui sistem e-medicine pentru analizarea și monitorizarea datelor epidemiologice, cu accent pe implementarea și analiza comparativă a modelului SIR în patru medii de

programare reprezentative pentru spectrul aplicațiilor de calcul științific: **R**, **Julia**, **Python** și **SAS**.

1.2 Obiectivele lucrării

Prezenta lucrare urmărește realizarea unui sistem coerent și cuprinzător, structurat în jurul mai multor obiective principale:

1. **Fundamentarea matematică riguroasă** a modelului SIR prin derivarea ecuațiilor din principii prime, demonstrarea proprietăților matematice fundamentale (existență, unicitate, pozitivitate, conservare) și analiza calitativă a soluțiilor (echilibre, stabilitate, numărul de reproducere de bază $\mathcal{R}_0$).

2. **Implementarea modelului SIR** în patru limbaje de programare — R, Julia, Python și SAS — utilizând metodele numerice adecvate pentru rezolvarea sistemelor de ecuații diferențiale ordinare (metoda Euler, Runge-Kutta de ordinul 4, metode adaptive).

3. **Evaluarea comparativă a performanței** implementărilor pe multiple configurații hardware, urmărind metricile de timp de execuție, consum de memorie și scalabilitate în funcție de dimensiunea setului de date.

4. **Proiectarea și implementarea infrastructurii distribuite** pentru stocarea datelor epidemiologice, bazată pe baza de date YugabyteDB rulată într-un cluster Kubernetes, asigurând disponibilitate ridicată și distribuție eficientă a datelor.

5. **Dezvoltarea aplicației web e-medicine**, compusă dintr-un API backend implementat în Go și o interfață frontend Vue.js, care integrează toate componentele sistemului și oferă o perspectivă unificată asupra rezultatelor.

1.3 Contribuții originale

Contribuțiile originale ale acestei lucrări pot fi sintetizate după cum urmează:

- **Analiză comparativă sistematică** a implementărilor modelului SIR în R, Julia, Python și SAS, acoperind atât corectitudinea numerică, cât și performanța computațională pe configurații hardware diverse.

- **Metodologie de benchmarking reproducibilă** pentru evaluarea performanței algoritmilor epidemiologici în limbaje de programare științifică, inclusiv protocoale de testare și metrici standardizate.

- **Arhitectură distribuită de stocare a datelor epidemiologice** bazată pe YugabyteDB și Kubernetes, cu schema relațională optimizată pentru interogări specifice domeniului epidemiologic.
- **Sistem web integrat** pentru vizualizarea și compararea rezultatelor modelării epidemiologice, combinând un API REST performant (Go) cu o interfață web modernă și reactivă (Vue.js).

1.4 Structura lucrării

Lucrarea este organizată în opt capitole, după cum urmează:

Capitolul 2 (*Epidemiologie Computațională și Modelarea Matematică*) prezintă fundamentele epidemiologiei matematice. Sunt introduse modelele compartimentale (SI, SIS, SIR, SEIR, SIRS), cu accent pe analiza matematică riguroasă a modelului SIR: derivare, teoreme de existență și unicitate, analiza stabilității, numărul de reproducere de bază $\mathcal{R}_0$ și ecuația dimensiunii finale. Capitolul include și o trecere în revistă a metodelor numerice utilizate pentru rezolvarea sistemelor de ecuații diferențiale ordinare.

Capitolul 3 (*Tehnologii și Instrumente Utilizate*) prezintă fundamentele teoretice și caracteristicile tehnice ale instrumentelor utilizate: limbajele de programare R, Julia, Python și SAS, baza de date distribuită YugabyteDB, sistemul de orchestrare Kubernetes, limbajul de programare Go și framework-ul frontend Vue.js.

Capitolul 4 (*Arhitectura Sistemului E-Medicine*) descrie arhitectura globală a sistemului, cerințele funcționale și non-funcționale, schema bazei de date distribuite, arhitectura API-ului backend și structura componentelor frontend.

Capitolul 5 (*Implementarea Modelului SIR*) detaliază implementările practice ale modelului SIR în fiecare limbaj de programare, inclusiv seturile de date utilizate, parametrii modelului și interfața de integrare cu API-ul Go.

Capitolul 6 (*Evaluarea Performanței și Scalabilității*) prezintă metodologia de benchmarking, configurațiile hardware utilizate și rezultatele comparative ale evaluărilor de performanță.

Capitolul 7 (*Aplicația Web E-Medicine*) detaliază implementarea aplicației web, documentând atât backend-ul Go, cât și frontend-ul Vue.js, cu descrierea funcționalităților și a fluxurilor de utilizare.

Capitolul 8 (*Concluzii și Direcții Viitoare*) sintetizează contribuțiile lucrării, formulează concluzii privind implementările și infrastructura, și identifică direcții de dezvoltare ulterioară.

Lucrarea se încheie cu **Bibliografie** și **Anexe** care conțin codul sursă complet al implementărilor, schema bazei de date, configurațiile Kubernetes și documentația API-ului REST.

Capitolul 2

Epidemiologie Computațională și Modelarea Matematică a Bolilor Infecțioase

2.1 Fundamente de epidemiologie

2.1.1 Definiții și concepte fundamentale

Epidemiologia este știința care studiază distribuția, determinanții și controlul stărilor sau evenimentelor legate de sănătate în populații definite [1]. În sens mai specific, *epidemiologia infecțioasă* se ocupă cu dinamica transmiterii bolilor cauzate de agenți patogeni biologici (virusuri, bacterii, paraziți) în populații umane sau animale.

Definiție 2.1 (Epidemic, pandemic, endemic)**.** O boală infecțioasă se numește **endemică** dacă persistă la nivel constant într-o regiune geografică definită. Se numește **epidemie** dacă numărul de cazuri depășește în mod semnificativ nivelul endemic așteptat, și **pandemie** dacă afectează mai multe continente sau o mare parte a globului.

Definiție 2.2 (Incidență și prevalență)**. Incidența** reprezintă numărul de cazuri noi ale unei boli apărute într-o populație de risc, într-un interval de timp specificat. Se exprimă ca rată:

$$\text{Incidență} = \frac{\text{cazuri noi în intervalul } [t_1, t_2]}{\text{populație la risc} \times (t_2 - t_1)}. \tag{2.1}$$

Prevalența reprezintă proporția indivizilor afectați la un moment dat:

$$\text{Prevalență} = \frac{\text{cazuri existente la momentul } t}{\text{populație totală la momentul } t}. \tag{2.2}$$

2.1.2 Parametrii fundamentali ai transmiterii

Definiție 2.3 (Intervalul serial și intervalul de generație)**. Intervalul serial** este durata medie dintre apariția simptomelor la cazul primar și la cazul secundar generat de acesta. **Intervalul de generație** este durata medie dintre infecția cazului primar și infecția unui caz secundar.

Definiție 2.4 (Perioada de incubație)**. Perioada de incubație** este intervalul de timp dintre momentul infectării și apariția primelor simptome. Este distinctă de *perioada de latență*, care reprezintă intervalul dintre infecție și debutul infecțiozității.

Definiție 2.5 (Numărul de reproducere de bază $\mathcal{R}_0$)**.** Numărul de reproducere de bază $\mathcal{R}_0$ este definit ca numărul mediu de infecții secundare produse de un singur individ infectat introdus într-o populație complet susceptibilă, pe durata perioadei sale infecțioase [4]. Este cel mai important parametru al epidemiologiei matematice, deoarece determină potențialul epidemic al unui agent patogen.

O consecință imediată a Teoremă 2.5 este că:

- dacă $\mathcal{R}_0 < 1$, fiecare caz infectat generează în medie mai puțin de un caz nou, astfel că epidemia se stinge spontan;
- dacă $\mathcal{R}_0 > 1$, fiecare caz generează în medie mai mult de un caz nou, epidemia se propagă exponențial în stadiul inițial;
- dacă $\mathcal{R}_0 = 1$, epidemia se menține la nivel endemic instabil.

Numărul de reproducere *efectiv* (sau în timp real) $\mathcal{R}(t)$ generalizează $\mathcal{R}_0$ pentru o populație parțial imunizată:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N}, \tag{2.3}$$

unde $S(t)$ este numărul de indivizi susceptibili la momentul t și N este mărimea populației totale.

2.2 Modele compartimentale în epidemiologie

2.2.1 Cadrul general

Modelele compartimentale reprezintă clasa fundamentală de modele matematice în epidemiologie [2]. Principiul lor constă în împărțirea populației în *compartimente* care descriu stări epidemiologice distincte, iar tranziția dintre compartimente este guvernată de un sistem de ecuații diferențiale ordinare (EDO).

Fie N mărimea totală a populației (presupusă constantă). Fiecare compartiment $X_i(t)$ reprezintă numărul de indivizi în starea i la momentul t, cu $\sum_i X_i(t) = N$. Dinamica tranzițiilor este descrisă de sistemul:

$$\frac{d\mathbf{x}}{dt} = \mathbf{f}(\mathbf{x}, t, \boldsymbol{\theta}), \quad \mathbf{x}(0) = \mathbf{x}_0, \tag{2.4}$$

unde $\mathbf{x} = (X_1, \ldots, X_n)^\intercal$, $\boldsymbol{\theta}$ este vectorul parametrilor și $\mathbf{f}$ descrie fluxurile nete dintre compartimente.

2.2.2 Modelul SI

Cel mai simplu model compartimental este modelul **SI** (Susceptibili-Infectați), care presupune că infecția este permanentă (nu există recuperare):

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N}, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N}. \end{cases} \tag{2.5}$$

Cu condiția $S + I = N$, sistemul (2.5) se reduce la o ecuație logistică în I, cu soluția analitică:

$$I(t) = \frac{N}{1 + \left(\frac{N - I_0}{I_0}\right) e^{-\beta t}}, \tag{2.6}$$

unde $I_0 = I(0)$. Toți indivizii sunt infectați pe termen lung ($I(\infty) = N$).

2.2.3 Modelul SIS

Modelul **SIS** (Susceptibili-Infectați-Susceptibili) extinde modelul SI prin includerea recuperării fără imunitate, permițând re-infectarea:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \gamma I, \\ \frac{dI}{dt} = \quad \frac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.7}$$

Parametrul $\gamma > 0$ reprezintă rata de recuperare ($1/\gamma$ este durata medie a perioadei infecțioase). Pentru modelul SIS, $\mathcal{R}_0 = \beta/\gamma$, iar:

- dacă $\mathcal{R}_0 \leq 1$: echilibrul fără boală $I^* = 0$ este stabil global;
- dacă $\mathcal{R}_0 > 1$: există un echilibru endemic stabil $I^* = N(1 - 1/\mathcal{R}_0) > 0$.

2.2.4 Modelul SEIR

Modelul **SEIR** introduce compartimentul E (Expuși / în perioadă de latență), care ține cont de faptul că indivizii infectați nu sunt imediat infecțioși:

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N}, \\ \frac{dE}{dt} = \frac{\beta SI}{N} - \sigma E, \\ \frac{dI}{dt} = \sigma E - \gamma I, \\ \frac{dR}{dt} = \gamma I, \end{cases} \tag{2.8}$$

unde $\sigma = 1/T_E$ este rata de tranziție din starea expusă în starea infecțioasă (T_E = durata medie de incubație). Numărul de reproducere de bază pentru modelul SEIR este $\mathcal{R}_0 = \beta/\gamma$, identic cu cel al modelului SIR.

2.2.5 Modelul SIRS

Modelul **SIRS** generalizează modelul SIR prin imunitate temporară (indivizii recuperați pot deveni din nou susceptibili după un interval de timp):

$$\begin{cases} \frac{dS}{dt} = -\frac{\beta SI}{N} + \xi R, \\ \frac{dI}{dt} = \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} = \gamma I - \xi R, \end{cases} \tag{2.9}$$

unde $\xi = 1/T_R$ este rata de pierdere a imunității (T_R = durata medie a imunității). Dacă $\mathcal{R}_0 > 1$, modelul SIRS admite un echilibru endemic stabil.

2.3 Modelul SIR — Analiză Matematică Detaliată

2.3.1 Ipoteze și derivarea ecuațiilor din principii prime

Modelul SIR, formulat de Kermack și McKendrick [3], se bazează pe o serie de ipoteze simplificatoare care trebuie înțelese explicit:

1. **Populație închisă și omogenă:** Nu există nașteri, decese naturale sau migrații; $N = \text{const}$. Fiecare individ are aceeași probabilitate de a intra în contact cu orice alt individ (amestecare omogenă).

2. **Transmitere prin acțiune de masă:** Rata de contact eficient (producătoare de infecție) este proporțională cu produsul $S \cdot I$.

3. **Imunitate permanentă:** Un individ recuperat nu poate fi re-infectat; compartimentul R este absorbant.

4. **Durata infecțioasă exponențial distribuită:** Rata de recuperare γ este constantă, iar durata medie a perioadei infecțioase este $1/\gamma$.

5. **Fără mortalitate indusă de boală:** Sau, echivalent, mortalitatea specifică bolii este neglijabilă față de dinamica transmiterii.

Derivarea termenului de infecție. Considerăm că în unitatea de timp, un individ susceptibil are în medie c contacte cu alți indivizi aleși uniform din populație. Probabilitatea ca un contact arbitrar să fie cu un individ infectat este I/N, iar probabilitatea de transmitere la contact eficient este p. Astfel, rata de infecție a unui susceptibil este $c \cdot p \cdot I/N = \beta I/N$, unde $\beta = cp$ este parametrul de transmitere. Fluxul total de noi infecții pe unitate de timp este $(\beta I/N) \cdot S$.

Sistemul de ecuații diferențiale ordinare al modelului SIR este:

$$\boxed{\begin{aligned} \frac{dS}{dt} &= -\frac{\beta SI}{N}, \\ \frac{dI}{dt} &= \frac{\beta SI}{N} - \gamma I, \\ \frac{dR}{dt} &= \gamma I, \end{aligned}} \tag{2.10}$$

cu condiții inițiale $S(0) = S_0 > 0$, $I(0) = I_0 > 0$, $R(0) = R_0 \geq 0$ și $S_0 + I_0 + R_0 = N$.

Diagrama de flux a compartimentelor este prezentată în Fig. 2.1.

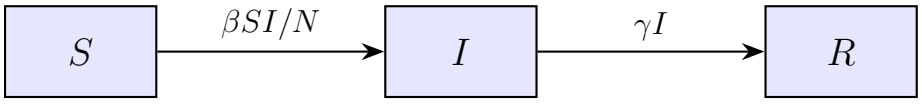


Figura 2.1: Diagrama de flux a compartimentelor modelului SIR. Fluxul $\beta SI/N$ reprezintă rata de noi infecții, iar γI reprezintă rata de recuperare.

2.3.2 Proprietăți matematice fundamentale

Înainte de a analiza comportamentul calitativ al soluțiilor, stabilim proprietățile matematice esențiale: existența și unicitatea soluției, conservarea populației totale și invarianța domeniului pozitiv.

Teoremă 2.6 (Existență și unicitate globală)**.** *Fie $\beta, \gamma > 0$, $N > 0$ și $(S_0, I_0, R_0) \in \mathbb{R}^3$ cu $S_0, I_0 > 0$, $R_0 \geq 0$. Atunci sistemul* (2.10) *admite o unică soluție globală $(S, I, R) \in C^1([0, \infty); \mathbb{R}^3)$.*

Demonstrație. Definim $\mathbf{x} = (S, I, R)^\intercal$ și $\mathbf{f}(\mathbf{x}) = (-\beta SI/N,\ \beta SI/N - \gamma I,\ \gamma I)^\intercal$. Funcția $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^3$ este de clasă C^∞ (polinomul de gradul doi în S și I este neted), în particular continuu diferențiabilă. Prin *teorema Picard-Lindelöf* (Cauchy-Lipschitz), există o soluție locală unică pe un interval $[0, T_{\max})$.

Existența *globală* ($T_{\max} = \infty$) se stabilește prin mărginirea soluției. Deoarece $S(t), I(t), R(t) \geq 0$ (Teorema 2.8) și $S + I + R = N$ (Teorema 2.7), rezultă că $\|\mathbf{x}(t)\| \leq N\sqrt{3}$ pentru orice $t \in [0, T_{\max})$. Soluția mărginită nu poate prezenta blow-up în timp finit, deci $T_{\max} = \infty$. □

Teoremă 2.7 (Conservarea populației totale)**.** *Pentru orice soluție a sistemului* (2.10)*,*

$$S(t) + I(t) + R(t) = N \quad \forall\, t \geq 0. \tag{2.11}$$

Demonstrație. Calculăm derivata sumei:

$$\begin{aligned} \frac{d}{dt}(S + I + R) &= \frac{dS}{dt} + \frac{dI}{dt} + \frac{dR}{dt} \\ &= -\frac{\beta SI}{N} + \left(\frac{\beta SI}{N} - \gamma I\right) + \gamma I \\ &= 0. \end{aligned}$$

Prin integrare, $S(t) + I(t) + R(t) = S_0 + I_0 + R_0 = N$. □

Teoremă 2.8 (Pozitivitate și mărginire)**.** *Dacă $S_0 > 0$, $I_0 > 0$, $R_0 \geq 0$, atunci:*

$$S(t) > 0, \quad I(t) > 0, \quad R(t) \geq 0, \quad \forall\, t > 0. \tag{2.12}$$

În particular, $S(t), I(t), R(t) \in [0, N]$ pentru orice $t \geq 0$.

Demonstrație. **(a) Pozitivitatea lui S:** Din ecuația $dS/dt = -(\beta I/N)S$, aceasta este o ecuație diferențială liniară omogenă în S cu coeficient $-\beta I(t)/N$. Soluția este:

$$S(t) = S_0 \cdot \exp\left(-\frac{\beta}{N}\int_0^t I(s)\, ds\right). \tag{2.13}$$

Deoarece integrala este finită și $S_0 > 0$, rezultă $S(t) > 0$ pentru orice $t \geq 0$ (presupunând $I(s) \geq 0$, demonstrat mai jos).

(b) Pozitivitatea lui I: Rescrierea ecuației a doua ca $dI/dt = (\beta S(t)/N - \gamma)\, I$

arată că aceasta este liniară în I:

$$I(t) = I_0 \cdot \exp\left(\int_0^t \left[\frac{\beta S(\tau)}{N} - \gamma\right] d\tau\right) > 0, \tag{2.14}$$

deoarece $I_0 > 0$ și exponențiala este strict pozitivă.

(c) Non-negativitatea lui R: Din $dR/dt = \gamma I(t) > 0$ și $R_0 \geq 0$, funcția R este strict crescătoare, deci $R(t) > R_0 \geq 0$.

Mărginirea urmează din Teorema 2.7: $S(t) \leq N$, $I(t) \leq N$, $R(t) \leq N$. □

Observație 2.9. Teorema 2.8 implică că domeniul pozitiv

$$\Omega = \{(S, I, R) \in \mathbb{R}^3_{\geq 0} : S + I + R = N\} \tag{2.15}$$

este *invariant pozitiv* față de fluxul sistemului (2.10). Prin urmare, este suficient să studiem comportamentul soluțiilor pe Ω, care este un simplex în $\mathbb{R}^3$.

Teoremă 2.10 (Monotonia compartimentelor)**.** *De-a lungul oricărei soluții a sistemului* (2.10)*:*

(i) $S(t)$ *este strict descrescătoare;*

(ii) $R(t)$ *este strict crescătoare;*

(iii) $I(t)$ *atinge un singur maxim în* $t^* = \inf\{t > 0 : S(t) = \gamma N/\beta\}$ *(dacă* $S_0 > \gamma N/\beta$*); altfel este descrescătoare.*

Demonstraţie. (i) $dS/dt = -\beta SI/N < 0$ deoarece $S > 0$, $I > 0$. *(ii)* $dR/dt = \gamma I > 0$. *(iii)* $dI/dt = (\beta S/N - \gamma)I > 0 \Leftrightarrow S > \gamma N/\beta$. Deoarece S este strict descrescătoare, există cel mult un moment t^* când $S(t^*) = \gamma N/\beta$, care este maximul lui I. □

2.3.3 Analiza echilibrelor și stabilitate

Punctele de echilibru ale sistemului (2.10) sunt obținute din condiția $\mathbf{f}(\mathbf{x}^*) = \mathbf{0}$. Ecuațiile conduc la:

$$-\frac{\beta S^* I^*}{N} = 0, \quad \frac{\beta S^* I^*}{N} - \gamma I^* = 0, \quad \gamma I^* = 0. \tag{2.16}$$

Din a treia ecuație, $I^* = 0$. Substituind, obținem că orice punct de forma $(S^*, 0, N - S^*)$ cu $S^* \in [0, N]$ este echilibru. Mulțimea echilibrelor este:

$$\mathcal{E} = \{(S^*, 0, N - S^*) : S^* \in [0, N]\} \subset \Omega. \tag{2.17}$$

Observație 2.11. Spre deosebire de modelele SIS și SIRS, modelul SIR nu admite echilibru endemic cu $I^* > 0$. Aceasta reflectă faptul că, pe termen lung, epidemia se stinge întotdeauna (deși poate infecta o fracție mare din populație înainte de stingere).

Analiza stabilității locale. Deoarece $R = N - S - I$, sistemul se reduce la două ecuații (eliminând R):

$$\begin{cases} \frac{dS}{dt} = -\dfrac{\beta SI}{N}, \\ \frac{dI}{dt} = \dfrac{\beta SI}{N} - \gamma I. \end{cases} \tag{2.18}$$

Jacobianul sistemului (2.18) evaluat la un echilibru $(S^*, 0)$ este:

$$J(S^*, 0) = \begin{pmatrix} 0 & -\beta S^*/N \\ 0 & \beta S^*/N - \gamma \end{pmatrix}. \tag{2.19}$$

Valorile proprii sunt $\lambda_1 = 0$ și $\lambda_2 = \beta S^*/N - \gamma$.

Teoremă 2.12 (Stabilitatea echilibrului fără boală)**.** *Echilibrul fără boală* $(S^*, 0)$ *este:*

- *stabil (nelinear) dacă* $\beta S^*/N < \gamma$*, adică dacă* $S^*/N < 1/\mathcal{R}_0$*;*
- *instabil dacă* $\beta S^*/N > \gamma$*.*

Demonstrație. Valorea proprie $\lambda_2 = \beta S^*/N - \gamma$ determină stabilitatea pe direcția I. Deoarece $\lambda_1 = 0$, analiza liniară este degenerată și trebuie completată cu argumente neliniare. Totuși, din Teorema 2.10, $I(t)$ este descrescătoare dacă $S(t) < \gamma N/\beta$ pentru tot $t > 0$, ceea ce se întâmplă când $S_0 < \gamma N/\beta$, implicând stingerea epidemiei. □

2.3.4 Numărul de reproducere de bază $\mathcal{R}_0$

Definiție prin matricea generației următoare

Metoda *matricei generației următoare* (next-generation matrix), formalizată de Diekmann, Heesterbeek și Metz [4] și extinsă de van den Driessche și Watmough [5], furnizează o procedură sistematică pentru calculul $\mathcal{R}_0$ în modele compartimentale de complexitate arbitrară.

Fie $\mathbf{x}_I$ vectorul compartimentelor infectate. Scriem:

$$\frac{d\mathbf{x}_I}{dt} = \mathcal{F}(\mathbf{x}) - \mathcal{V}(\mathbf{x}), \tag{2.20}$$

unde $\mathcal{F}_i$ reprezintă rata de apariție a noilor infecții în compartimentul i, iar $\mathcal{V}_i$ reprezintă rata netă de ieșire din compartimentul i prin toate celelalte procese.

Matricele Jacobiene evaluate la echilibrul fără boală $\mathbf{x}_0$ sunt:

$$F = \left[\frac{\partial \mathcal{F}_i}{\partial x_j}\right]_{\mathbf{x}_0}, \quad V = \left[\frac{\partial \mathcal{V}_i}{\partial x_j}\right]_{\mathbf{x}_0}. \tag{2.21}$$

Matricea generației următoare este $K = FV^{-1}$, iar:

$$\mathcal{R}_0 = \rho(K), \tag{2.22}$$

unde $\rho(\cdot)$ este raza spectrală (cea mai mare valoare proprie în modul).

Calculul $\mathcal{R}_0$ pentru modelul SIR

Pentru modelul SIR, singurul compartiment infectat este $\{I\}$. La echilibrul fără boală $(S, I, R) = (N, 0, 0)$:

$$\mathcal{F}(I) = \frac{\beta SI}{N} \implies F = \left[\frac{\partial \mathcal{F}}{\partial I}\right]_{S=N} = \frac{\beta N}{N} = \beta. \tag{2.23}$$

$$\mathcal{V}(I) = \gamma I \implies V = \gamma. \tag{2.24}$$

Matricea K este scalarul:

$$K = FV^{-1} = \frac{\beta}{\gamma}. \tag{2.25}$$

Prin urmare:

$$\boxed{\mathcal{R}_0 = \frac{\beta}{\gamma}.} \tag{2.26}$$

Interpretarea biologică este directă: $1/\gamma$ este durata medie a perioadei infecțioase, iar β este numărul mediu de contacte eficiente pe unitate de timp; produsul lor dă numărul de infecții secundare produse pe durata infecțiozității.

Teoremă 2.13 (Pragul epidemic)**.** *Considerăm o populație inițial complet susceptibilă ($S_0 \approx N$). Atunci:*

(i) Dacă $\mathcal{R}_0 > 1$, numărul de infectați crește inițial exponențial și are loc o epidemie ($I(t)$ atinge un maxim $I_{\max} > I_0$).

(ii) Dacă $\mathcal{R}_0 \leq 1$, $I(t)$ este imediat descrescătoare și nu are loc nicio epidemie.

Demonstraţie. Din $dI/dt|_{t=0} = I_0(\beta S_0/N - \gamma) \approx I_0(\beta - \gamma)$ (pentru $S_0 \approx N$), concluzia urmează din semnul expresiei $\beta - \gamma$:

$$\left.\frac{dI}{dt}\right|_{t=0} > 0 \iff \beta > \gamma \iff \frac{\beta}{\gamma} > 1 \iff \mathcal{R}_0 > 1. \tag{2.27}$$

□

2.3.5 Pragul de imunizare în turmă (herd immunity)

Definiție 2.14 (Imunitate de grup)**.** Imunitatea de grup (*herd immunity*) se referă la protecția indirectă a persoanelor susceptibile, generată atunci când o fracție suficient de mare din populație este imunizată (fie natural, fie prin vaccinare), astfel încât transmiterea susținută a agentului patogen devine imposibilă.

Epidemia se stinge când numărul de reproducere efectiv devine subunitar:

$$\mathcal{R}(t) = \mathcal{R}_0 \cdot \frac{S(t)}{N} < 1 \iff \frac{S(t)}{N} < \frac{1}{\mathcal{R}_0}. \tag{2.28}$$

Dacă o fracție p din populație este imunizată (cu $S/N = 1 - p$), condiția de blocare a epidemiei devine:

$$(1-p) \cdot \mathcal{R}_0 < 1 \iff p > 1 - \frac{1}{\mathcal{R}_0}. \tag{2.29}$$

Definiție 2.15 (Pragul de imunizare în turmă)**.** Fracția minimă a populației care trebuie imunizată pentru a preveni propagarea epidemiei este:

$$p_c = 1 - \frac{1}{\mathcal{R}_0}. \tag{2.30}$$

Exemplu 2.16. Pentru rujeolă, $\mathcal{R}_0 \approx 12$–18, deci $p_c \approx 92$–94%. Pentru COVID-19 (tulpina originală), $\mathcal{R}_0 \approx 2$–3, deci $p_c \approx 50$–67%. Variantele ulterioare (Delta, Omicron) au $\mathcal{R}_0$ semnificativ mai mare, crescând corespunzător pragul de imunizare.

2.3.6 Ecuația dimensiunii finale a epidemiei

O caracteristică remarcabilă a modelului SIR este că permite calculul analitic al *dimensiunii finale* a epidemiei — fracția totală din populație care este infectată în cursul epidemiei — fără a rezolva explicit sistemul diferențial.

Teoremă 2.17 (Ecuația dimensiunii finale)**.** *Fie $z = 1 - S(\infty)/N$ fracția din populație infectată cumulativ pe durata epidemiei, presupunând $S_0 = N$ și $R_0 = 0$. Atunci z este unica rădăcină pozitivă a ecuației transcendentale:*

$$z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.31}$$

Demonstrație. Împărțind ecuația pentru S la ecuația pentru R:

$$\frac{dS}{dR} = \frac{-\beta SI/N}{\gamma I} = -\frac{\beta}{\gamma N} S = -\frac{\mathcal{R}_0}{N} S. \tag{2.32}$$

Această ecuație separabilă în S și R se integrează direct:

$$\int_{S_0}^{S(t)} \frac{dS'}{S'} = -\frac{\mathcal{R}_0}{N} \int_0^{R(t)} dR' \implies \ln \frac{S(t)}{S_0} = -\frac{\mathcal{R}_0}{N} R(t). \tag{2.33}$$

Deci $S(t) = S_0 \exp(-\mathcal{R}_0 R(t)/N)$.

La $t \to \infty$, $I(\infty) = 0$ (demonstrat mai jos), deci $R(\infty) = N - S(\infty)$ (pentru

$R_0 = 0$). Substituind cu $S_0 = N$:

$$S(\infty) = Ne^{-\mathcal{R}_0(N - S(\infty))/N}. \tag{2.34}$$

Notând $z = 1 - S(\infty)/N$ și împărțind prin N:

$$1 - z = e^{-\mathcal{R}_0 z} \implies z = 1 - e^{-\mathcal{R}_0 z}. \tag{2.35}$$

Că $I(\infty) = 0$: Funcția $R(t)$ este monoton crescătoare și mărginită de N, deci $\lim_{t\to\infty} R(t)$ există. Dacă $I(\infty) = \ell > 0$, atunci $R'(t) = \gamma I(t) \geq \gamma\ell > 0$ pentru t suficient de mare, ceea ce implică $R(t) \to \infty$, contradicție. Deci $I(\infty) = 0$.

Unicitatea rădăcinii pozitive: Fie $g(z) = 1 - e^{-\mathcal{R}_0 z} - z$. Avem $g(0) = 0$, $g'(0) = \mathcal{R}_0 - 1 > 0$ (dacă $\mathcal{R}_0 > 1$), și $g(1) = 1 - e^{-\mathcal{R}_0} - 1 = -e^{-\mathcal{R}_0} < 0$. Deoarece $g''(z) = -\mathcal{R}_0^2 e^{-\mathcal{R}_0 z} < 0$, funcția g este concavă, deci are exact o singură rădăcină în $(0, 1)$. □

Observație 2.18. Ecuația (2.31) este *transcendentală* și nu admite soluție în formă închisă pentru $\mathcal{R}_0 \neq 1$. Se rezolvă numeric, de exemplu prin metoda Newton-Raphson sau prin iterație de punct fix.

Corolar 2.19 (Rata de atac finală)**.** *Dacă $\mathcal{R}_0 > 1$, rata de atac finală z^* satisface $z^* \in (0, 1)$ și crește monoton cu $\mathcal{R}_0$. Limita $\lim_{\mathcal{R}_0\to\infty} z^* = 1$.*

2.3.7 Extensii ale modelului SIR

Modelul SIR de bază poate fi extins în multiple direcții pentru a acomoda realități biologice mai complexe:

- **Demografie:** includerea nașterilor și mortalității naturale permite studiul echilibrelor endemice pe termen lung.
- **Structura pe vârste:** matricele de contact structurate pe grupe de vârstă (modele de tip POLYMOD) capturează heterogenitatea contactelor sociale.
- **Spațialitate:** modelele cu patch-uri sau ecuații diferențiale parțiale descriu propagarea geografică.
- **Stochasticitate:** modelele stochastice (proces de ramificare, lanțuri Markov în timp continuu) sunt esențiale pentru populații mici.
- **Control:** vaccinarea, izolarea și tratamentul pot fi modelate ca fluxuri suplimentare între compartimente.

2.4 Metode Numerice pentru Ecuații Diferențiale Ordinare

2.4.1 Clasificarea metodelor numerice

Soluția analitică a sistemului SIR nu există în formă închisă (cu excepția cazurilor degenerate), ceea ce face indispensabilă rezolvarea numerică. O metodă numerică pentru problema cu valori inițiale (PVI):

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y}), \quad \mathbf{y}(t_0) = \mathbf{y}_0, \tag{2.36}$$

generează o secvență de aproximații $\mathbf{y}_n \approx \mathbf{y}(t_n)$ la punctele $t_n = t_0 + nh$.

Metodele numerice se clasifică după mai multe criterii:

- **Explicite vs. implicite:** la metodele explicite, $\mathbf{y}_{n+1}$ se calculează direct din valorile anterioare; la cele implicite, apare $\mathbf{y}_{n+1}$ și în membrul drept (sistem nonlinear de rezolvat la fiecare pas).
- **Un-pas vs. multi-pas:** metodele un-pas folosesc doar $\mathbf{y}_n$ pentru a calcula $\mathbf{y}_{n+1}$; metodele multi-pas folosesc mai mulți pași anteriori.
- **Ordin de convergență:** ordinul p al metodei determină că eroarea globală $\|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^p)$.
- **Pas fix vs. adaptiv:** metodele adaptive ajustează h dinamic pe baza unei estimări a erorii locale.

2.4.2 Metoda Euler explicită

Metoda Euler (1768) este cea mai simplă metodă numerică un-pas, obținută prin truncherea seriei Taylor la ordinul întâi:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \cdot \mathbf{f}(t_n, \mathbf{y}_n). \tag{2.37}$$

Teoremă 2.20 (Eroarea globală a metodei Euler)**.** *Dacă* $\mathbf{f}$ *este Lipschitz în* $\mathbf{y}$ *cu constanta* L *și* $\mathbf{y} \in C^2([t_0, T])$*, eroarea globală a metodei Euler satisface:*

$$\max_{n:t_n \le T} \|\mathbf{y}(t_n) - \mathbf{y}_n\| \le \frac{h}{2L} \max_{t \in [t_0,T]} \|\mathbf{y}''(t)\| \left(e^{L(T-t_0)} - 1\right) = \mathcal{O}(h). \tag{2.38}$$

Eroarea locală de trunchiere (pe un singur pas) este $\mathcal{O}(h^2)$, deci metoda are ordinul local 2 și ordinul global 1.

Aplicată sistemului SIR cu $\mathbf{y}_n = (S_n, I_n, R_n)^\intercal$, schema Euler este:

Algoritm 2.1 Metoda Euler pentru sistemul SIR

```
Input:  beta, gamma, N, S0, I0, R0, t0, tf, h
Output: Secventa (tn, Sn, In, Rn) pentru tn in [t0, tf]

(S, I, R) <- (S0, I0, R0);  t <- t0
while t < tf do
    dS <- -beta * S * I / N
    dI <-  beta * S * I / N - gamma * I
    dR <-  gamma * I
    S  <- S + h * dS
    I  <- I + h * dI
    R  <- R + h * dR
    t  <- t + h
    Inregistreaza (t, S, I, R)
end while
```

Stabilitate. Analiza stabilității metodei Euler pe ecuația test $y' = \lambda y$ ($\lambda \in \mathbb{C}$) conduce la factorul de amplificare $z = 1 + h\lambda$. Metoda este stabilă dacă $|1 + h\lambda| \leq 1$, ceea ce definește domeniul de stabilitate (disc de rază 1 centrat în -1 în planul $h\lambda$). Această restricție limiteaza pasul de integrare h, în special pentru probleme *stiff*.

2.4.3 Metoda Runge-Kutta de ordinul 4 (RK4)

Metodele Runge-Kutta sunt metode un-pas explicite care realizează mai multe evaluări ale funcției $\mathbf{f}$ la fiecare pas, atingând ordine superioare fără a recurge la derivate de ordin înalt ale soluției.

Derivare și formulă

Metoda RK4 clasică (cu 4 stadii) este dată de:

$$\begin{aligned} \mathbf{k}_1 &= h \cdot \mathbf{f}(t_n, \mathbf{y}_n), \\ \mathbf{k}_2 &= h \cdot \mathbf{f}\big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_1}{2}\big), \\ \mathbf{k}_3 &= h \cdot \mathbf{f}\big(t_n + \tfrac{h}{2},\ \mathbf{y}_n + \tfrac{\mathbf{k}_2}{2}\big), \\ \mathbf{k}_4 &= h \cdot \mathbf{f}(t_n + h,\ \mathbf{y}_n + \mathbf{k}_3), \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{1}{6}\left(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\right). \end{aligned} \tag{2.39}$$

Coeficienții metodei sunt reprezentați compact prin *tabloul Butcher*:

$$\begin{array}{c|cccc} 0 & & & & \\ 1/2 & 1/2 & & & \\ 1/2 & 0 & 1/2 & & \\ 1 & 0 & 0 & 1 & \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array} \tag{2.40}$$

Teoremă 2.21 (Ordinul metodei RK4)**.** *Metoda* (2.39) *are ordinul de convergență* $p = 4$*: dacă* $\mathbf{f} \in C^4$ *și* $\mathbf{y} \in C^5([t_0, T])$*, atunci eroarea globală satisface:*

$$\max_n \|\mathbf{y}(t_n) - \mathbf{y}_n\| = \mathcal{O}(h^4). \tag{2.41}$$

Schiță de demonstrație. Eroarea locală de trunchiere τ_n se obține substituind soluția exactă $\mathbf{y}(t_n)$ în schema (2.39) și expandând în serie Taylor. Se poate arăta că primele 4 condiții de ordin (condițiile de arbore Butcher pentru $p = 4$) sunt satisfăcute de coeficienții (2.40), implicând $\tau_n = \mathcal{O}(h^5)$. Eroarea globală este $\mathcal{O}(h^4)$ prin acumularea $\sim 1/h$ de pași. Detalii complete în [6]. □

Algoritm 2.2 Metoda Runge-Kutta de ordinul 4 pentru sistemul SIR

```
Input:  f, y0, t0, tf, h
Output: Secventa (tn, yn)

y <- y0;  t <- t0
while t < tf do
    k1 <- h * f(t,        y           )
    k2 <- h * f(t + h/2, y + k1/2     )
    k3 <- h * f(t + h/2, y + k2/2     )
    k4 <- h * f(t + h,   y + k3       )
    y  <- y + (k1 + 2*k2 + 2*k3 + k4) / 6
    t  <- t + h
end while
```

2.4.4 Metode Runge-Kutta adaptive (RK45 / Dormand-Prince)

Metodele cu pas fix impun utilizatorului alegerea unui h adecvat, care este o sarcină dificilă în practică. Metodele *adaptive* ajustează automat pasul h pe baza unei estimări a erorii locale, asigurând că eroarea rămâne sub o toleranță prestabilită ε.

Metode Runge-Kutta încorporate

O *pereche RK încorporată* de ordine p și $p+1$ calculează doi estimatori pentru $\mathbf{y}_{n+1}$ folosind *același* set de evaluări:

$$\hat{\mathbf{y}}_{n+1} = \mathbf{y}_n + h\sum_{i=1}^{s} b_i \mathbf{k}_i \quad \text{(ordinul } p\text{)}, \tag{2.42}$$

$$\tilde{\mathbf{y}}_{n+1} = \mathbf{y}_n + h\sum_{i=1}^{s} \tilde{b}_i \mathbf{k}_i \quad \text{(ordinul } p+1\text{)}, \tag{2.43}$$

cu $\mathbf{k}_i = \mathbf{f}(t_n + c_i h, \mathbf{y}_n + h\sum_{j<i} a_{ij}\mathbf{k}_j)$. Estimatorul de eroare locală este:

$$\mathbf{e}_{n+1} = \tilde{\mathbf{y}}_{n+1} - \hat{\mathbf{y}}_{n+1} = h\sum_{i=1}^{s}(b_i - \tilde{b}_i)\mathbf{k}_i. \tag{2.44}$$

Metoda Dormand-Prince (DOPRI5 / RK45)

Perechea **Dormand-Prince** [7], implementată ca `ode45` în MATLAB, `RK45` în SciPy (Python), `Tsit5` în Julia și `lsoda`/`ode45` în R (pachetul `deSolve`), utilizează 6 evaluări de funcție și are ordinul principal 5 cu estimator de eroare de ordinul 4.

Tabloul Butcher complet al metodei Dormand-Prince este:

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	−56/15	32/9				
8/9	19372/6561	−25360/2187	64448/6561	−212/729			
1	9017/3168	−355/33	46732/5247	49/176	−5103/18656		
1	35/384	0	500/1113	125/192	−2187/6784	11/84	
$\hat{b}$	35/384	0	500/1113	125/192	−2187/6784	11/84	0
$\tilde{b}$	5179/57600	0	7571/16695	393/640	−92097/339200	187/2100	1/40

(2.45)

Control adaptiv al pasului

Pasul h este actualizat după fiecare pas pe baza normei erorii:

$$\text{err} = \left\| \frac{\mathbf{e}_{n+1}}{\text{atol} + \text{rtol} \cdot |\mathbf{y}_{n+1}|} \right\|_{\infty}, \tag{2.46}$$

unde atol și rtol sunt toleranțe absolute și relative. Pasul următor este calculat prin formula:

$$h_{\text{nou}} = h \cdot \min\big(f_{\max},\ \max\big(f_{\min},\ S \cdot \text{err}^{-1/(p+1)}\big)\big)\,, \tag{2.47}$$

cu factorul de siguranță $S = 0.9$ și limitele $f_{\min} = 0.1,\ f_{\max} = 10$.

2.4.5 Stabilitate numerică și probleme stiff

Definiție 2.22 (Problemă stiff)**.** O problemă cu valori inițiale (2.36) este numită *stiff* dacă valorile proprii ale Jacobianului $\partial\mathbf{f}/\partial\mathbf{y}$ au componente cu parte reală negativă de magnitudine mare, impunând pași de integrare h extrem de mici pentru metodele explicite din motive de stabilitate, chiar dacă soluția variază lent.

Aplicație la modelul SIR. Jacobianul sistemului SIR (2.18) este:

$$J(S, I) = \begin{pmatrix} -\beta I/N & -\beta S/N \\ \beta I/N & \beta S/N - \gamma \end{pmatrix}. \tag{2.48}$$

Valorile proprii depind de (S, I). La debutul epidemiei ($S \approx N$, I mic), valorile proprii sunt aproximativ 0 și $\beta - \gamma$, deci problema nu este stiff. Modelul SIR de bază nu este stiff pentru parametri tipici epidemiologici. Totuși, extensiile cu mai multe compartimente și rate foarte diferite pot produce probleme stiff, necesitând metode implicite (Runge-Kutta implicit, BDF).

Observație 2.23. Metodele implicite (de exemplu, metoda Euler implicită, Crank-Nicolson, Radau) au domenii de stabilitate nelimitate și sunt preferate pentru probleme stiff, în ciuda costului computațional superior (rezolvarea unui sistem nonlinear la fiecare pas).

2.5 Date Epidemiologice — Surse și Preprocesare

2.5.1 Surse de date epidemiologice

Implementarea și validarea modelelor epidemiologice computaționale necesită date empirice de calitate. Principalele surse de date epidemiologice utilizate în lucrare sunt:

- **WHO (Organizația Mondială a Sănătății):** rapoarte de situație, baze de date privind mortalitatea și morbiditatea, date de supraveghere a bolilor infecțioase.
- **ECDC (Centrul European pentru Prevenirea și Controlul Bolilor):** date de supraveghere pentru statele membre UE, inclusiv serii temporale zilnice pentru COVID-19 și alte boli infecțioase.

- **Johns Hopkins University CSSE:** setul de date COVID-19 (cazuri confirmate, decese, recuperați) la nivel de țară și regiune, disponibil public pe GitHub.
- **Our World in Data:** date armonizate din surse multiple, incluzând rate de vaccinare, indicatori de mobilitate și date socioeconomice.

2.5.2 Structura datelor și preprocesare

Datele epidemiologice brute prezintă probleme tipice care necesită preprocesare:

- **Valori lipsă:** interpolate liniar sau propagate din ultimul valor disponibil (forward fill);
- **Raportări retrospective:** corecții retroactive care produc valori negative în seriile de incidență zilnică;
- **Efecte de weekend:** ciclicitate săptămânală în raportare, uniformizată prin media mobilă de 7 zile;
- **Inconsistențe geografice:** modificări de granițe sau reorganizări administrative.

2.5.3 Estimarea parametrilor din date reale

Parametrii β și γ ai modelului SIR pot fi estimați din date observate prin metode statistice:

- **Cel mai mic pătrat neliniar (NLS):** minimizarea erorii pătratice dintre datele observate și soluția numerică a modelului;
- **Estimarea maximă a verosimilității (MLE):** presupunând un model de zgomot (Poisson sau binomial negativ pentru date de incidență), se maximizează funcția de log-verosimilitate;
- **Inferență bayesiană (MCMC):** permite cuantificarea incertitudinii parametrilor prin distribuții posterioare.

Estimatorul $\hat{\gamma}$ poate fi obținut direct dacă durata medie a perioadei infecțioase este disponibilă din studii clinice: $\hat{\gamma} = 1/T_{\text{infecțios}}$. Parametrul β este mai dificil de estimat și depinde de comportamentul social, intervenții și variante ale agentului patogen.

Capitolul 3

Tehnologii și Instrumente Utilizate

3.1 Limbaje de programare pentru calcul științific

Alegerea limbajului de programare pentru implementarea modelelor matematice nu este neutră: performanța, expresivitatea și ecosistemul de biblioteci au un impact direct asupra eficienței dezvoltării și calității rezultatelor. Această secțiune prezintă fundamentele și caracteristicile tehnice ale celor patru limbaje utilizate.

3.1.1 R

Istoric și filosofie

R este un limbaj de programare și mediu software pentru calcul statistic și grafică, descendent direct al limbajului S (Bell Laboratories, 1976). A fost creat de Ross Ihaka și Robert Gentleman la Universitatea din Auckland în 1993, iar în 1997 a fost adoptat ca proiect open-source de *R Development Core Team* [8]. Filosofia R este centrată pe date: vectorii, matricile și cadrele de date (data frames) sunt cetățeni de primă clasă, iar gramatica limbajului este proiectată pentru manipularea și analiza datelor.

Caracteristici tehnice relevante

R este un limbaj *interpretat*, dinamic tipizat, cu suport pentru programare funcțională, orientată pe obiecte (sistemele S3, S4 și R5/R6) și procedurală. Caracteristici relevante pentru calculul științific:

- **Vectorizare nativă:** operațiile aritmetice și logice aplicate pe vectori și matrice sunt automat vectorizate și executate prin cod C/Fortran compilat, evitând buclele interpretate costisitoare.
- **Lazy evaluation:** argumentele funcțiilor sunt evaluate doar când sunt necesare, permițând construcții expresive.

- **Gestionarea memoriei:** R utilizează un garbage collector cu copiere la scriere (copy-on-write), care poate genera overhead semnificativ pentru structuri de date mari.
- **Interfețe C/Fortran:** prin funcțiile `.C()`, `.Call()` și `.Fortran()`, R poate apela cod compilat nativ, stând la baza multor pachete de înaltă performanță.

Pachete relevante pentru modelul SIR

- **deSolve** [6]: pachetul principal pentru rezolvarea sistemelor de EDO, ecuații diferențiale cu derivate parțiale (EDP) și ecuații algebrico-diferențiale (EAD). Integrează solver-e consacrate: `lsoda` (Adams/BDF adaptiv), `ode45` (Dormand-Prince), `rk4` (Runge-Kutta fix). Interfața principală este funcția `ode()`.
- **ggplot2**: sistem de grafică bazat pe *grammar of graphics*, utilizat pentru vizualizarea traiectoriilor SIR.
- **data.table**: manipulare eficientă (în memorie) a seturilor de date mari, cu sintaxă concisă.
- **microbenchmark** și **bench**: măsurarea precisă a timpilor de execuție pentru benchmarking.

3.1.2 Julia

Istoric și filosofie

Julia a fost creat la MIT de Jeff Bezanson, Alan Edelman, Stefan Karpinski și Viral B. Shah, cu versiunea 1.0 lansată în august 2018 [9]. Motivația proiectului a fost depășirea *problemei celor două limbaje* (two-language problem): cercetătorii scriu prototipuri în R/Python (productivitate ridicată, viteză scăzută) și re-implementează codul critic în C/Fortran (viteză ridicată, productivitate scăzută). Julia vizează ambele obiective simultan.

Compilare Just-In-Time și sistemul de tipuri

Julia utilizează compilare **JIT** (Just-In-Time) bazată pe LLVM, cu o particularitate fundamentală: specializarea pe tipuri concrete. Când o funcție este apelată pentru prima dată cu argumente de tipuri specifice, compilatorul generează cod mașină optimizat pentru acele tipuri exacte. Apelurile ulterioare cu aceleași tipuri rulează direct codul compilat, apropiind performanța de C.

- **Multiple dispatch:** funcțiile pot fi definite cu comportamente diferite pentru combinații diferite de tipuri ale argumentelor (metode). Compilatorul selectează și compilează metoda corectă la momentul apelului.
- **Sistem de tipuri parametric:** tipurile pot fi parametrizate (ex. `Array{Float64, 1}`), permițând generarea eficientă de cod specializat.
- **Metaprogramming:** macrocomenzile Julia (`@macro`) permit generarea și transformarea codului la momentul parsării.

DifferentialEquations.jl

Ecosistemul **SciML** (Scientific Machine Learning) al Julia conține pachetul `DifferentialEquations.j` [10], cel mai cuprinzător framework open-source pentru rezolvarea ecuațiilor diferențiale. Acesta include:

- Peste 300 de algoritmi de rezolvare pentru EDO, EDP, EAD, ecuații diferențiale stochastice (EDS) și ecuații cu întârziere (DDEs);
- Solver-e de înaltă performanță: `Tsit5` (Tsitouras RK 4/5), `Vern7/9` (Verner 7/9), `Rodas4/5` (pentru probleme stiff);
- Integrare nativă cu GPU-uri și calcul distribuit;
- Diferențiere automată (AD) pentru estimarea parametrilor și identificarea sistemelor.

3.1.3 Python

Istoric și filosofie

Python a fost creat de Guido van Rossum și lansat în 1991 [11]. Filosofia sa, rezumată în *Zen of Python* (`PEP 20`), accentuează lizibilitatea și simplitatea codului. Python a devenit limbajul dominant în știința datelor și machine learning datorită ecosistemului său extins și comunității masive.

Caracteristici tehnice și ecosistem științific

Python este un limbaj interpretat, dinamic tipizat, multi-paradigmă. Principala limitare pentru calcul intensiv este **GIL** (Global Interpreter Lock), care previne execuția paralelă a thread-urilor Python pure. Performanța este recuperată prin biblioteci care rulează cod nativ C/Fortran în afara GIL:

- **NumPy**: calculul numeric vectorizat pe array-uri dense; operațiile sunt implementate în C/Fortran și nu sunt supuse GIL.

- **SciPy** [12]: algoritmică științifică, incluzând modulul `scipy.integrate` cu funcțiile `odeint` (wrapper LSODA) și `solve_ivp` (interfață modernă cu multiple metode: RK45, DOP853, Radau, BDF, LSODA).
- **Matplotlib**: vizualizare 2D/3D flexibilă.
- **Pandas**: manipularea datelor tabulare.

Funcțiile de rezolvare a EDO în SciPy

Interfața `solve_ivp` din `scipy.integrate` este apelată astfel:

```python
from scipy.integrate import solve_ivp
import numpy as np

def sir_model(t, y, beta, gamma, N):
    S, I, R = y
    dSdt = -beta * S * I / N
    dIdt =  beta * S * I / N - gamma * I
    dRdt =  gamma * I
    return [dSdt, dIdt, dRdt]

# Parametri
N = 1_000_000; beta = 0.3; gamma = 0.1
S0, I0, R0 = N - 1, 1, 0
t_span = (0, 160); t_eval = np.linspace(0, 160, 1601)

sol = solve_ivp(
    fun=lambda t, y: sir_model(t, y, beta, gamma, N),
    t_span=t_span,
    y0=[S0, I0, R0],
    method='RK45',
    t_eval=t_eval,
    rtol=1e-8,
    atol=1e-10
)
```

Listing 3.1: Interfața solve_ivp din SciPy pentru rezolvarea sistemului SIR

3.1.4 SAS

Istoric și model de licențiere

SAS (Statistical Analysis System) a fost dezvoltat la North Carolina State University începând cu 1966 și comercializat de SAS Institute din 1976. Spre deosebire de R, Julia și Python, SAS este un software proprietar cu licențiere comercială, larg utilizat în industria farmaceutică, financiară și în cercetarea clinică, în parte datorită cerințelor reglementare (FDA validation, CFR 21 Part 11).

PROC IML pentru calcul numeric

PROC IML (Interactive Matrix Language) este procedura SAS care permite programare matriceală generală, analogă cu limbajele de tip MATLAB. Prin IML, sistemul SIR poate fi rezolvat implementând manual schema RK4:

```
proc iml;
  start sir_rhs(t, y, parms);
    beta = parms[1]; gamma = parms[2]; N = parms[3];
    S = y[1]; I = y[2]; R = y[3];
    dSdt = -beta * S * I / N;
    dIdt =  beta * S * I / N - gamma * I;
    dRdt =  gamma * I;
    return (dSdt || dIdt || dRdt);
  finish;

  start rk4_step(t, y, h, parms);
    k1 = h * sir_rhs(t,       y,           parms);
    k2 = h * sir_rhs(t + h/2, y + k1/2,   parms);
    k3 = h * sir_rhs(t + h/2, y + k2/2,   parms);
    k4 = h * sir_rhs(t + h,   y + k3,     parms);
    return (y + (k1 + 2*k2 + 2*k3 + k4) / 6);
  finish;

  /* Parametri si conditii initiale */
  N = 1000000; beta = 0.3; gamma = 0.1;
  parms = beta || gamma || N;
  y = (N-1) || 1 || 0;
  t = 0; h = 0.1; tf = 160;

  results = j(0, 4);
  do while (t <= tf);
    results = results // (t || y);
```

```
    y = rk4_step(t, y, h, parms);
    t = t + h;
  end;
quit;
```

Listing 3.2: Rezolvarea sistemului SIR în SAS/IML cu schema RK4

Comparație cu alternativele open-source

SAS prezintă avantaje în medii de producție reglementate (trasabilitate, audit trail, validare formală), dar dezavantaje semnificative pentru cercetare exploratorie: cost de licențiere ridicat, ciclu de actualizare lent și un ecosistem de pachete mult mai restrâns comparativ cu R și Python. Varianta **SAS OnDemand for Academics** oferă acces gratuit pentru uz educațional.

3.1.5 Analiză comparativă a limbajelor

Table 3.1 sintetizează principalele caracteristici ale celor patru limbaje din perspectiva utilizării pentru implementarea modelului SIR.

Tabela 3.1: Comparație a limbajelor de programare utilizate

Criteriu	**R**	**Julia**	**Python**	**SAS**
Licență	Open-source	Open-source	Open-source	Proprietar
Paradigmă	Funcț./OOP	Multi	Multi	Procedural
Compilare	Interpretat	JIT (LLVM)	Interpretat	Interpretat
Solver EDO	deSolve	DiffEq.jl	SciPy	PROC IML
Performanță rel.	Medie	Înaltă	Medie	Medie
Maturitate ecos.	Înaltă	Medie	Înaltă	Înaltă
Utilizare acad.	Ridicată	Medie	Ridicată	Ridicată

3.2 Baze de date distribuite — YugabyteDB

3.2.1 Concepte de baze de date distribuite

O *bază de date distribuită* este o colecție de date distribuite fizic pe mai multe noduri (mașini sau locații geografice), gestionată de un sistem software care oferă transparența localizării: aplicațiile interacționează cu baza de date ca și cum ar fi centralizată.

Definiție 3.1 (Teorema CAP)**.** Teorema CAP (Brewer, 2000) afirmă că un sistem distribuit nu poate garanta simultan mai mult de două dintre următoarele trei proprietăți:

- **Consistency (C):** toate nodurile văd aceleași date în același moment (citirile reflectă întotdeauna ultima scriere confirmată);
- **Availability (A):** orice cerere validă primește un răspuns (eventual fără ultima versiune a datelor);
- **Partition tolerance (P):** sistemul continuă să funcționeze chiar dacă comunicarea între noduri este parțial întreruptă.

Bazele de date distribuite moderne (YugabyteDB, CockroachDB, Spanner) optează pentru **CP** (consistency + partition tolerance), sacrificând disponibilitatea perfectă în favoarea coerenței datelor.

3.2.2 Arhitectura YugabyteDB

YugabyteDB [13] este o bază de date distribuită open-source, construită pentru a combina caracteristicile PostgreSQL cu scalabilitatea orizontală și toleranța la erori a sistemelor NoSQL. Arhitectura sa se bazează pe doi piloni principali:

1. Stratul de stocare DocDB. DocDB este motorul de stocare distribuit al YugabyteDB, bazat pe arhitectura **LSM-tree** (Log-Structured Merge-tree) derivată din **RocksDB**. Datele sunt stocate în *tablete* — unități de sharding distribuite pe noduri. Fiecare tabletă este replicată pe cel puțin trei noduri prin **protocolul Raft** (consens distribuit):

- Un nod este ales *lider*; scrierile trec prin lider;
- Leaderul replică modificarea pe nodurile urmașe (*followers*);
- O scriere este confirmată după ce o *majoritate* (quorum) de noduri o confirmă.

2. Nivelurile de interfață API.

- **YSQL** (YugabyteDB SQL): compatibil cu PostgreSQL 11+, suportând SQL complet, tranzacții ACID distribuite, join-uri, indecși, triggere și proceduri stocate;
- **YCQL** (YugabyteDB CQL): compatibil cu Apache Cassandra, optimizat pentru workload-uri de tip wide-column cu disponibilitate extinsă.

3.2.3 Sharding și replicare

YugabyteDB suportă două strategii de sharding:

- **Hash sharding:** cheia primară este hașurată și distribuită uniform pe tablete; asigură echilibru de sarcină dar nu suportă eficient *range scans*;

- **Range sharding:** tabletele acoperă intervale contigue de valori ale cheii; eficient pentru interogări de tip range, dar poate produce *hot spots*.

Observație 3.2. Pentru datele epidemiologice cu interogări frecvente pe intervale de date (serii temporale), range sharding pe coloana `timestamp` sau `date` este strategia recomandată, împreună cu un index secundar pe codul bolii și/sau regiune geografică.

3.3 Orchestrarea containerelor — Kubernetes

3.3.1 Containere și nevoia de orchestrare

Containerizarea (Docker) permite împachetarea unei aplicații împreună cu toate dependențele sale într-o unitate portabilă și reproductibilă. Pe măsură ce sistemele cresc în complexitate și numărul de containere depășește câteva zeci, gestionarea manuală devine infezabilă: unde rulează fiecare container? Cum se reporneşte dacă cade? Cum se scalează?

Kubernetes (K8s) [14, 15] este platforma open-source de orchestrare a containerelor, inițial dezvoltată la Google pe baza experienței cu sistemul intern Borg, și donată CNCF (Cloud Native Computing Foundation) în 2014.

3.3.2 Arhitectura Kubernetes

Un cluster Kubernetes este compus din:

Control Plane:

- **kube-apiserver:** componenta centrală, expune API-ul K8s (REST/gRPC); toate celelalte componente comunică exclusiv prin API server;
- **etcd:** baza de date distribuită key-value (Raft-based) care stochează starea întregului cluster;
- **kube-scheduler:** atribuie pod-uri nodurilor disponibile pe baza resurselor disponibile și a constrângerilor de afinitate;
- **kube-controller-manager:** rulează controllere care reconciliază starea dorită cu starea actuală (Deployment, ReplicaSet, StatefulSet etc.).

Worker Nodes:

- **kubelet:** agent pe fiecare nod, gestionează ciclul de viață al pod-urilor (Pull image, start/stop/restart containere);
- **kube-proxy:** gestionează regulile de rețea (iptables/IPVS) pentru accesul la servicii;

- **Container runtime:** interfața CRI (containerd, CRI-O).

3.3.3 Resurse Kubernetes relevante

- **Pod:** unitatea atomică de deployment; unul sau mai multe containere cu spațiu de rețea și stocare partajate;
- **Deployment:** gestionează un set de pod-uri identice; asigură restart automat și rolling updates;
- **StatefulSet:** similar Deployment, dar menține identitate stabilă pentru pod-uri (hostname, persistent volume); *esențial* pentru YugabyteDB;
- **Service:** abstractizare pentru accesul la un set de pod-uri; asigură load balancing și descoperire de servicii;
- **PersistentVolume / PersistentVolumeClaim:** stocare persistentă independentă de ciclul de viață al pod-ului;
- **ConfigMap / Secret:** configurare și date sensibile externalizate.

3.4 Backend API — Go

3.4.1 Caracteristici și filosofia limbajului Go

Go (cunoscut și ca Golang) este un limbaj de programare open-source creat la Google de Robert Griesemer, Rob Pike și Ken Thompson, cu prima versiune stabilă în 2012 [16]. Go a fost proiectat pentru productivitate la scară mare, cu accent pe simplitate, compilare rapidă și concurență structurată.

Caracteristici relevante pentru construcția unui API backend:

- **Compilare nativă rapidă:** Go compilează direct la cod mașin, producând binare statice fără dependențe externe;
- **Goroutine și canale (CSP):** concurența este modelată prin goroutine (fire de execuție ușoare, 2KB stack inițial) și canale (typed, pentru comunicare sincronă/asincronă); modelul Communicating Sequential Processes (CSP, Hoare 1978) este preferat față de memoria partajată cu mutex;
- **Interfețe structurale (duck typing):** un tip implementează implicit o interfață dacă definește metodele necesare, fără declarație explicită;
- **Garbage collection:** GC concurrent cu latență redusă ($<$ 1ms stop-the-world în versiunile recente);

- **Standard library bogată:** pachetul `net/http` suportă HTTP/1.1, HTTP/2 și TLS out-of-the-box, fără framework extern.

3.4.2 Ecosistem pentru API-uri REST

Deși `net/http` din biblioteca standard este suficient pentru API-uri simple, framework-uri ca **Gin** sau **Echo** adaugă:

- Router performant (radix tree pentru $\mathcal{O}(\log k)$ matching);
- Middleware pipeline (logging, CORS, rate limiting, autentificare JWT);
- Binding și validare automată a parametrilor HTTP (JSON, query params, path params);
- Generare automată a documentației OpenAPI/Swagger.

3.5 Frontend — Vue.js

3.5.1 Arhitectura Vue 3 și Composition API

Vue.js [17] este un framework JavaScript progresiv pentru construirea interfețelor utilizator, creat de Evan You în 2014. Vue 3 (2020) introduce **Composition API**, o alternativă la Options API care permite organizarea logicii prin funcționalitate (nu prin tip — data, methods, computed) și reutilizarea logicii prin *composables*.

Arhitectura Vue 3 se bazează pe:

- **Reactivity system:** bazat pe `Proxy` ES6, permite urmărirea dependențelor reactive cu granularitate fină;
- **Virtual DOM:** reconciliere eficientă a arborelui DOM cu algoritmul de diferențiere O(n);
- **Single File Components (SFC):** fișiere `.vue` care încapsulează template (HTML), script (JS/TS) și style (CSS) într-o unitate coezivă.

3.5.2 Ecosistem și biblioteci utilizate

- **Vue Router:** rutare client-side cu suport pentru rute dinamice, navigare programatică și guards de autentificare;
- **Pinia:** state management bazat pe Composition API, mai simplu decât Vuex și mai bine integrat cu TypeScript;

- **Axios:** client HTTP cu interceptori, pentru apelurile REST la API-ul Go;
- **ECharts / Chart.js:** vizualizare date epidemiologice (grafice SIR, comparații de performanță, hărți de densitate).

Capitolul 4

Arhitectura Sistemului E-Medicine

4.1 Vedere de ansamblu

Diagramă arhitecturală globală a sistemului: cluster K8s cu YugabyteDB, runner-e SIR (R, Julia, Python, SAS), API Go, frontend Vue.js. Descrie fluxul de date de la ingestia datelor epidemiologice până la vizualizare.

4.2 Cerințe funcționale și non-funcționale

4.2.1 Cerințe funcționale

Lista completă de cerințe funcționale (use cases): (1) Încărcare seturi de date epidemiologice, (2) Declanșare rulare model SIR într-un limbaj ales, (3) Vizualizare rezultate și grafice, (4) Comparare rezultate între limbaje, (5) Export rezultate (CSV, PDF).

4.2.2 Cerințe non-funcționale

Cerințe non-funcționale: performanță (latența API), disponibilitate (SLA YugabyteDB pe K8s), scalabilitate (număr simultan de rulări), securitate (autentificare JWT), extensibilitate (adăugare noi limbaje).

4.3 Arhitectura bazei de date distribuite

4.3.1 Schema relațională

Diagrama ERD completă. Tabelele principale: `datasets` (metadate seturi de date), `epidemiological_records` (valori S/I/R per timestamp și regiune), `simulation_runs` (parametri și metadate ale rulărilor), `simulation_results` (rezultatele S(t), I(t), R(t) per rulare), `benchmarks` (timpii de execuție per configurație hardware), `languages` (R, Julia, Python, SAS).

4.3.2 Modelul datelor epidemiologice

Descrierea schemei de date pentru seria temporală epidemiologică: granularitate (zilnică/săptămânală), acoperire geografică, câmpuri obligatorii vs. opționale. Justificare alegeri de indexare.

4.3.3 Strategia de partitionare și replicare în YugabyteDB

Range sharding pe (disease_code, date) pentru interogări de serii temporale. Replicare Raft cu factor 3. Configurarea tablet-elor și tuning al compaction-ului RocksDB.

4.4 Arhitectura API-ului backend Go

4.4.1 Structura modulară a proiectului

Structura directorului Go: cmd/, internal/(api, db, runner, models), pkg/(sirmath). Descriere pattern-uri arhitecturale: repository pattern, service layer, dependency injection.

4.4.2 Diagrama endpointurilor REST

Tabel complet al endpointurilor REST: GET /api/datasets, POST /api/datasets, POST /api/simulations (trigger rulare), GET /api/simulations/id/results, GET /api/benchmarks, GET /api/health. Documentație OpenAPI.

4.4.3 Integrarea cu runner-ele de algoritmi

Cum apelează API-ul Go implementările SIR: subprocess cu stdin/stdout JSON, sau binare native? Gestiunea asincronă a rulărilor (goroutines, job queue).

4.5 Arhitectura frontend Vue.js

4.5.1 Structura componentelor

Arborele de componente Vue: layout-uri, pagini (Dashboard, Datasets, Simulations, Benchmarks, Compare), componente reutilizabile (SIRChart, BenchmarkTable, ParameterForm).

4.5.2 Fluxul de date și state management

Diagrama fluxului de date: Pinia stores (datasetsStore, simulationsStore, benchmarkStore), apeluri Axios la API-ul Go, reactive updates în componente.

4.6 Infrastructura Kubernetes

4.6.1 Topologia cluster-ului

Diagrama cluster-ului K8s: master node(s), worker nodes, noduri YugabyteDB (StatefulSet x3). Specifică configurația hardware minimă recomandată.

4.6.2 Deployment YugabyteDB pe Kubernetes

Helm chart YugabyteDB, StatefulSet configuration, PersistentVolumeClaims, Services (headless service pentru DNS intern, LoadBalancer pentru accesul extern).

4.6.3 Deployment API Go și frontend Vue.js

Deployment și Service pentru API Go (containerizat cu Dockerfile multi-stage), Deployment Nginx pentru frontend Vue.js, Ingress controller (nginx-ingress sau Traefik) pentru rutarea traficului extern.

Capitolul 5

Implementarea Modelului SIR

5.1 Seturile de date utilizate

Descrierea seturilor de date: surse (WHO/ECDC/JHU), perioade, boli modelate (COVID-19, gripă sezonieră, rujeolă). Procesul de preprocesare și ingestie în YugabyteDB. Parametrii estimați (β, γ, N) pentru fiecare set de date.

5.2 Implementarea în R

5.2.1 Structura proiectului și dependențe

Structura directorului R, fișierul `renv.lock`/`DESCRIPTION`, dependențe: deSolve, ggplot2, jsonlite, optparse.

5.2.2 Rezolvarea numerică cu pachetul deSolve

Codul complet pentru definirea sistemului SIR, apelul `ode()` cu metodele lsoda, rk4 și ode45. Tratarea ieșirilor și exportul în format JSON pentru API-ul Go.

5.2.3 Estimarea parametrilor din date reale

Funcția de minimizare least-squares, apelul `optim()`, validarea pe date reale COVID-19.

5.2.4 Vizualizare și export rezultate

Grafice ggplot2 ale traiectoriilor S(t), I(t), R(t). Exportul datelor în format JSON și CSV.

5.3 Implementarea în Julia

5.3.1 Structura proiectului și pachete

Project.toml, Manifest.toml, dependențe: DifferentialEquations.jl, JSON3.jl, ArgParse.jl, Plots.jl.

5.3.2 DifferentialEquations.jl — definiție și solver-e

Codul Julia: definire ODEProblem, apelul `solve()` cu Tsit5(), Vern7(), RK4(). Compararea solver-elor din perspectiva acurateței și a vitezei. Utilizarea `@time` și `BenchmarkTools.jl`.

5.3.3 Optimizări specifice limbajului Julia

Evitarea alocărilor prin SVector/MVector (StaticArrays.jl), precompilare (`PackageCompiler.jl`), paralelism cu Threads.@threads pentru rulări multiple.

5.4 Implementarea în Python

5.4.1 Structura proiectului și dependențe

requirements.txt, structura pachetului Python: scipy, numpy, matplotlib, pandas, click.

5.4.2 scipy.integrate.solve_ivp

Codul complet, compararea metodelor: RK45, DOP853, Radau, BDF. Benchmarking cu `timeit` și `memory_profiler`.

5.4.3 Vectorizare și optimizare NumPy

Implementarea manuală a RK4 vectorizat cu NumPy față de solve_ivp. Analiza trade-off performanță/flexibilitate.

5.5 Implementarea în SAS

5.5.1 PROC IML — implementare completă

Codul SAS/IML complet pentru RK4 manual și RK45 manual. Exportul rezultatelor cu PROC EXPORT. Rutine de benchmarking cu `DATETIME()`.

5.5.2 Particularități față de celelalte implementări

Limitări SAS/IML: lipsa solver-elor adaptive native, overhead al mediului SAS, integrarea cu macro-urile SAS pentru parametrizare.

5.6 Interfața comună de integrare cu API-ul Go

Protocolul de comunicare: API-ul Go lansează un subprocess (R/Julia/Python/SAS), transmite parametrii ca JSON pe stdin, primește rezultatele ca JSON pe stdout. Schema JSON comună pentru input și output. Tratarea erorilor și timeout-urilor.

Capitolul 6

Evaluarea Performanței și Scalabilității

6.1 Metodologia de benchmarking

6.1.1 Configurații hardware utilizate

Tabel cu configurațiile hardware: laptop (Intel i5, 8GB RAM), workstation (AMD Ryzen 9, 32GB RAM), server cloud (AWS EC2 c5.4xlarge). Sistemul de operare, versiunile limbajelor, versiunile solver-elor.

6.1.2 Metrici urmărite

Metrici: (1) Timp de execuție wall-clock (mediana și IQR peste 30 rulări), (2) CPU time (user + system), (3) Peak memory usage (RSS), (4) Acuratețe numerică (eroarea față de soluția de referință DOPRI8 cu toleranță 10^{-12}).

6.1.3 Protocoale de testare și reproductibilitate

Protocol: warm-up runs, izolare procese, seed fix, versioning al codului de benchmark. Toate configurațiile rulate de 30 de ori, cu colectarea automatizată a rezultatelor.

6.2 Scalabilitate în funcție de dimensiunea setului de date

Experimente cu $N \in \{10^3, 10^4, 10^5, 10^6, 10^7\}$. Grafice log-log ale timpului de execuție față de N pentru fiecare limbaj. Identificarea complexității empirice.

6.3 Comparație timp de execuție — R vs Julia vs Python vs SAS

Tabel comparativ și grafice boxplot. Analiza statistică: test Kruskal-Wallis, comparații post-hoc Dunn pentru detectarea diferențelor semnificative. Discutarea overhead-ului de start-up al fiecărui limbaj față de timpul de calcul pur.

6.4 Consum de memorie și resurse

Profile de memorie pentru fiecare implementare. Grafice de utilizare a memoriei în timp. Identificarea cauzelor alocărilor excesive.

6.5 Acuratețea numerică a rezultatelor

Compararea soluțiilor numerice cu soluția de referință. Eroarea L^∞ și L^2 pentru $S(t)$, $I(t)$, $R(t)$. Verificarea conservării $S + I + R = N$ (eroarea de conservare).

6.6 Performanța bazei de date distribuite

6.6.1 Latență la scriere și citire

Măsurarea latenței P50/P95/P99 pentru operații INSERT și SELECT pe tabelele epidemiologice. Comparație cu PostgreSQL standalone.

6.6.2 Comportament sub load — teste de stress

Teste de load cu `pgbench` și/sau `cassandra-stress`. Throughput (tranzacții/secundă) față de numărul de clienți concurenți. Comportamentul clusterului la oprirea unuia din cele 3 noduri YugabyteDB.

6.7 Performanța API-ului Go

Benchmarking API cu `wrk` sau `hey`: RPS (requests/second), latență P95/P99, comportament sub concurență (10, 50, 100, 500 clienți simultani).

6.8 Analiză comparativă sintetică

Tabel de scoruri normalizate per metrică și per limbaj. Radar chart (spider chart) cu performanța relativă. Recomandări pentru scenarii de utilizare: volum mic vs. mare, precizie vs. viteză.

Capitolul 7

Aplicația Web E-Medicine

7.1 Descrierea funcționalităților aplicației

Prezentare generală a aplicației: ecranele principale, fluxurile de utilizare cheie. Screenshot-uri ale interfeței finale.

7.2 Backend Go — Implementare detaliată

7.2.1 Structura proiectului Go

Arborele de directoare al proiectului Go, convenții de denumire, separarea responsabilităților pe pachete: api, db, runner, models, config.

7.2.2 Implementarea endpointurilor REST

Codul Go pentru endpointurile principale, utilizând framework-ul ales (Gin/Echo). Demonstrarea pattern-ului: handler -> service -> repository -> DB.

7.2.3 Gestionarea execuției algoritmilor

Implementarea runner-elor asincrone: pool de goroutine, channel-uri pentru rezultate, context cancellation pentru timeout-uri. Codul Go pentru lansarea subprocess-urilor R, Julia, Python, SAS.

7.2.4 Conexiunea la YugabyteDB

Driverul `pgx` pentru PostgreSQL/YSQL, connection pooling, query-uri parametrizate pentru prevenirea SQL injection.

7.2.5 Autentificare și securitate

JWT middleware, CORS configuration, rate limiting, validarea input-ului, HTTPS cu certificate TLS.

7.3 Frontend Vue.js — Implementare detaliată

7.3.1 Structura componentelor și pagini

Arborele de componente, structura directorului `src/`, tipare de compunere (composables reutilizabile).

7.3.2 Dashboard epidemiologic

Componenta Dashboard: metrici cheie (R0 curent, vârf de infectați, dimensiunea finală estimată), grafice de sumar, ultimele rulări.

7.3.3 Vizualizarea graficelor SIR

Componenta SIRChart cu ECharts/Chart.js: afișarea traiectoriilor S(t), I(t), R(t), zoom interactiv, export PNG/SVG. Codul Vue 3 (Composition API).

7.3.4 Comparatorul de rezultate între limbaje

Pagina de comparare: suprapunerea graficelor din rulări multiple (R, Julia, Python, SAS pe același set de date), tabelul de diferențe numerice, graficul timpilor de execuție.

7.3.5 Interfața pentru seturile de date

CRUD pentru seturile de date: upload CSV, previzualizare, editarea metadatelor, ștergere. Validarea client-side a formularelor.

7.4 Fluxuri de utilizare (use cases)

Descrierea pas-cu-pas și screenshot-uri pentru: (1) Încărcarea unui set de date nou, (2) Declanșarea și monitorizarea unei simulări, (3) Compararea rezultatelor între limbaje, (4) Exportul rezultatelor.

7.5 Testare și validare funcțională

Unit teste Go (testing package), integration tests (testcontainers), end-to-end tests (Playwright/Cypress pentru Vue.js). Acoperire de cod.

Capitolul 8

Concluzii și Direcții de Dezvoltare Viitoare

8.1 Sinteza contribuțiilor

Rezumarea contribuțiilor principale ale lucrării: implementările comparative, infrastructura distribuită, aplicația web. Ce a fost atins față de obiectivele din Capitolul 1.

8.2 Concluzii privind implementările modelului SIR

Care limbaj s-a dovedit cel mai performant și în ce condiții? Care oferă cel mai bun compromis performanță/ușurință de utilizare? Recomandări pentru practicieni.

8.3 Concluzii privind infrastructura distribuită

Evaluarea YugabyteDB + K8s pentru date epidemiologice: avantaje (scalabilitate, fault tolerance), dezavantaje (complexitate operațională). Potrivirea arhitecturii pentru sistemele e-medicine.

8.4 Limitări ale lucrării

Limitări recunoscute: modelul SIR de bază ignoră structura de vârstă, spațialitatea și stochasticitatea; benchmarkurile sunt limitate la o serie de configurații hardware; interfața web nu implementează autentificare completă în versiunea prezentată.

8.5 Direcții de dezvoltare viitoare

Direcții: (1) Extinderea la modele SEIR, SIRS, modele stochastice; (2) Integrarea de modele de machine learning pentru estimarea parametrilor; (3) Adăugarea de vizualizări spațiale (hărți de propagare); (4) Suport pentru date în timp real (streaming cu Apache Kafka); (5) Implementarea unui modul de alertare epidemiologică automată; (6) Extinderea benchmarkurilor la GPU (CUDA, ROCm) pentru Julia și Python.

Bibliografie

[1] Roy M. Anderson and Robert M. May. *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press, Oxford, 1991.

[2] Herbert W. Hethcote. The mathematics of infectious diseases. *SIAM Review*, 42(4):599–653, 2000.

[3] W. O. Kermack and A. G. McKendrick. A contribution to the mathematical theory of epidemics. *Proceedings of the Royal Society of London. Series A*, 115(772):700–721, 1927.

[4] Odo Diekmann, Johan A. P. Heesterbeek, and Johan A. J. Metz. On the definition and the computation of the basic reproduction ratio $\mathcal{R}_0$ in models for infectious diseases in heterogeneous populations. *Journal of Mathematical Biology*, 28(4):365–382, 1990.

[5] Pauline van den Driessche and James Watmough. Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission. *Mathematical Biosciences*, 180(1–2):29–48, 2002.

[6] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*, volume 8 of *Springer Series in Computational Mathematics*. Springer, Berlin, 2 edition, 1993.

[7] John R. Dormand and Peter J. Prince. A family of embedded Runge-Kutta formulae. *Journal of Computational and Applied Mathematics*, 6(1):19–26, 1980.

[8] R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2024.

[9] Jeff Bezanson, Alan Edelman, Stefan Karpinski, and Viral B. Shah. Julia: A fresh approach to numerical computing. *SIAM Review*, 59(1):65–98, 2017.

[10] Christopher Rackauckas and Qing Nie. DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia. *Journal of Open Research Software*, 5(1):15, 2017.

[11] Guido Van Rossum and Fred L. Drake. *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA, 2009.

[12] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[13] Yugabyte, Inc. YugabyteDB architecture documentation. Documentație tehnică, 2024.

[14] Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes. *ACM Queue*, 14(1):70–93, 2016.

[15] The Kubernetes Authors. Kubernetes documentation. Documentație tehnică, 2024.

[16] The Go Authors. The Go programming language specification. Specificație oficială, 2024.

[17] Evan You et al. Vue.js documentation. Documentație oficială, 2024.

Anexa A

Cod Sursă — Implementarea R

Codul R complet pentru modelul SIR: definire sistem EDO, apel deSolve, estimare parametri, export JSON. Inclus prin `\lstinputlisting{...}` din fișierele sursă.

Anexa B

Cod Sursă — Implementarea Julia

Codul Julia complet: ODEProblem, DifferentialEquations.jl, benchmarking cu BenchmarkTools.jl, export JSON3.

Anexa C

Cod Sursă — Implementarea Python

Codul Python complet: scipy.integrate.solve_ivp, implementare manuală RK4 NumPy, benchmarking cu timeit, export JSON.

Anexa D

Cod Sursă — Implementarea SAS

Codul SAS/IML complet: PROC IML cu RK4 manual, PROC EXPORT, rutine de benchmarking.

Anexa E

Schema Bazei de Date

Diagrama ERD completă și scripturile SQL de creare a tabelelor (CREATE TABLE, indecși, constrângeri). Incluse ca listing-uri SQL.

Anexa F

Configurații Kubernetes

Manifestele YAML complete: StatefulSet YugabyteDB, Deployment API Go, Deployment frontend Nginx, Services, PVCs, ConfigMaps, Secrets, Ingress.

Anexa G

Documentația API REST

Specificația OpenAPI 3.0 (YAML) a API-ului Go sau echivalentul documentat manual: fiecare endpoint cu metoda HTTP, URL, parametrii, corpul cererii (request body), răspunsuri posibile (2xx, 4xx, 5xx) și exemple JSON.